

GPUMODE Lecture Study Notes: Lecture 27

gpu.cpp

Core Problem the Lecture Addresses

This lecture introduces `gpu.cpp`, a small C++ library for writing portable local GPU compute programs through WebGPU. The central problem is that personal devices contain large amounts of useful GPU compute, but direct access to that compute is fragmented across hardware vendors, operating systems, and low-level APIs.

CUDA gives excellent control on NVIDIA GPUs, but it is not portable to Apple GPUs, most phone GPUs, or many integrated GPUs. Metal is strong on Apple platforms, DirectX is tied to Windows, and Vulkan is portable but verbose. Raw WebGPU is more approachable than Vulkan, but still exposes enough setup machinery that it can feel designed for browser engines, game engines, and large runtime authors rather than small research programs.

The lecture's main engineering question is:

How can local GPU compute become easy enough to use from a small C++ program while still exposing kernels, buffers, workgroups, synchronization, memory reuse, and performance-relevant details?

The goal of `gpu.cpp` is not to create a full machine learning framework. It is a lightweight direct-compute layer that makes portable GPU programming feel closer to ordinary systems programming.

Required Background

Local GPU Compute

Local GPU compute refers to computation on devices such as laptops, phones, tablets, desktops, browsers, and local workstations. These devices are often treated as small compared with data center GPUs, but their aggregate compute can be enormous.

If there are N_{devices} deployed devices and each has approximate throughput P_{device} , the aggregate theoretical throughput is

$$P_{\text{aggregate}} = N_{\text{devices}} P_{\text{device}}.$$

The difficult part is not only the theoretical throughput. The difficult part is making that compute accessible across heterogeneous devices with different vendors, drivers, APIs, and capabilities.

Local compute is valuable for:

- low-latency audio, video, augmented reality, and robotics;
- privacy-sensitive applications in medicine, law, and personal computing;
- robustness when cloud endpoints fail;
- custom model runtimes with fine-grained control;
- browser-based GPU applications;
- interactive GPU programming education.

The CUDA Mental Model

CUDA programs are organized around grids, thread blocks, threads, warps, and memory hierarchy. A typical CUDA launch looks like this:

```
kernel<<<num_blocks, threads_per_block>>>(args);
```

Inside a CUDA kernel, a common one-dimensional global index is computed as follows:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

This maps each GPU thread to a logical element of the problem. WebGPU uses different terminology, but the same basic data-parallel idea appears through workgroups and invocations.

WebGPU

WebGPU is a portable GPU API specification. Despite the name, it is not only for browser graphics. It can also be used for native compute applications through implementations such as Dawn and wgpu.

A useful portability model is

$$\text{WebGPU} \longrightarrow \begin{cases} \text{Metal on Apple platforms,} \\ \text{DirectX on Windows,} \\ \text{Vulkan on Linux and other platforms.} \end{cases}$$

Dawn is Google’s native C++ implementation of WebGPU. It is used inside Chrome, but it can also be used outside the browser by native C++ programs.

This matters because `gpu.cpp` can use WebGPU without forcing the user to build a full web application.

WGSL

WGSL stands for WebGPU Shading Language. It is the device-side language used to write WebGPU compute kernels. `gpu.cpp` does not invent this language; it uses the language already defined by the WebGPU standard.

Important WGSL concepts include:

- storage buffers, which are similar to global memory buffers;
- workgroups, which are similar to CUDA thread blocks;
- invocations, which are similar to CUDA threads;
- workgroup memory, which is similar to CUDA shared memory;
- barriers, which synchronize invocations within a workgroup.

Main Concepts

What `gpu.cpp` Adds

The library provides a small C++ interface for the repetitive host-side pieces of GPU programming:

1. create a GPU context;
2. allocate GPU buffers;
3. create kernels from WGSL source;
4. bind buffers to kernels;
5. dispatch kernels asynchronously;
6. wait for completion when necessary;
7. copy data between CPU and GPU memory.

The key design goal is to remove non-algorithmic boilerplate while preserving a direct view of GPU execution.

Three Parts of a GPU Program

The lecture separates a compute workload into three parts.

Device Code

Device code is the WGSL program that runs on the GPU. It describes what each invocation does.

Resource Setup

Resource setup is host-side C++ code that creates buffers, kernels, bindings, and other objects required before dispatch.

Hot Path Dispatch

The hot path is the repeated section of the program. It should avoid unnecessary allocation, kernel creation, and CPU-GPU synchronization. Ideally, it mostly launches already-created kernels.

A simple runtime model is

$$T_{\text{total}} = T_{\text{setup}} + N_{\text{dispatch}} T_{\text{dispatch}} + T_{\text{movement}}.$$

Here, T_{setup} is setup time, N_{dispatch} is the number of kernel dispatches, T_{dispatch} is the average dispatch cost, and T_{movement} is the cost of CPU-GPU data movement.

Core Host-Side Abstractions

Context

A `Context` represents access to the GPU device and queue. It hides verbose adapter, device, and queue setup.

$$\text{Context} \approx \text{device handle} + \text{queue} + \text{runtime state}.$$

Tensor

A `Tensor` is a lightweight wrapper around GPU memory. In this lecture, it should not be confused with a full PyTorch tensor. It is closer to a device buffer with shape and element metadata.

For a row-major matrix with shape (M, N) , the linear offset of element (i, j) is

$$\text{offset}(i, j) = iN + j.$$

Kernel

A `Kernel` represents a WGSL compute program together with its resource bindings and pipeline state.

$$\text{Kernel} \approx \text{WGSL source} + \text{buffer bindings} + \text{compute pipeline}.$$

The main rule is:

Create kernels outside the hot path. Dispatch kernels inside the hot path.

Asynchronous Dispatch

WebGPU dispatch is asynchronous. The CPU submits work to the GPU and can continue executing. It only blocks if the program explicitly waits for completion or reads back data.

A poor synchronization pattern is

dispatch $\rightarrow$ wait $\rightarrow$ dispatch $\rightarrow$ wait.

A better pattern, when dependencies allow it, is

dispatch $\rightarrow$ dispatch $\rightarrow$ dispatch $\rightarrow$ wait once.

This keeps the GPU queue fuller and reduces unnecessary CPU-side blocking.

Hot Reloading

Because WGSL kernels can be stored as strings, an application can reload GPU kernel code at runtime. The lecture demonstrates this idea with interactive shader-like animations and GPU puzzles.

Hot reloading is valuable for:

- debugging indexing logic;
- teaching GPU programming interactively;
- changing simulation rules without restarting the program;
- experimenting with different kernels quickly;
- building browser-based GPU education tools.

Hardware-Level Explanation

Workgroups and Invocations

In CUDA, a thread block is a group of threads that can cooperate through shared memory. In WebGPU, the analogous concept is a workgroup. A workgroup contains many invocations.

A representative WGSL compute kernel has this shape:

```
@compute @workgroup_size(256)
fn main(@builtin(global_invocation_id) gid : vec3<u32>) {
    let i = gid.x;
}
```

The conceptual mapping is

CUDA thread block $\approx$ WebGPU workgroup,

and

CUDA thread $\approx$ WebGPU invocation.

If a vector has N elements and the workgroup size is B , the number of workgroups should be

$$G = \left\lceil \frac{N}{B} \right\rceil.$$

The total number of launched invocations is

$$N_{\text{launched}} = GB.$$

Since N_{launched} can be larger than N , kernels need bounds checks.

Memory Hierarchy

Even through a portable API, GPU performance is still controlled by the memory hierarchy:

- global memory is large but high latency;
- caches reduce repeated global memory traffic;
- workgroup memory is fast on-chip storage shared within a workgroup;
- registers are private to each invocation;
- barriers are needed when invocations share workgroup memory.

A central performance quantity is arithmetic intensity:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

Low arithmetic intensity usually indicates a bandwidth-bound kernel. High arithmetic intensity gives the kernel a better chance of becoming compute-bound.

Memory Coalescing

Memory coalescing means neighboring invocations access neighboring memory addresses. For a vector map operation,

$$\text{invocation } i \rightarrow x_i,$$

and

$$\text{invocation } i + 1 \rightarrow x_{i+1}.$$

This pattern is efficient because the memory system can combine or streamline adjacent accesses. Strided or random memory access patterns reduce effective bandwidth.

Portability Versus Vendor-Specific Features

The lecture emphasizes a trade-off. CUDA exposes NVIDIA-specific hardware features quickly. WebGPU must specify features in a portable way across multiple backends and devices. Therefore, features such as tensor cores may lag behind vendor-specific APIs.

The trade-off is

$$\text{CUDA} \rightarrow \text{maximum NVIDIA-specific control},$$

while

$$\text{WebGPU} \rightarrow \text{broader portability}.$$

Mathematical Reasoning

One-Dimensional Indexing

For a one-dimensional data-parallel kernel, the global index can be written as

$$i = gB + t,$$

where g is the workgroup index, B is the workgroup size, and t is the local invocation index.

A bounds-safe kernel should only operate when

$$0 \leq i < N.$$

Two-Dimensional Indexing

For a row-major matrix with N columns, element (r, c) is stored at

$$\text{offset}(r, c) = rN + c.$$

This formula appears in matrix multiplication, image processing, and tensor layout code.

Vector Map

A map operation applies a function independently to each element:

$$y_i = f(x_i).$$

Because each output depends only on one input, this operation is embarrassingly parallel.

Dot Product

A dot product computes

$$s = \sum_{k=0}^{K-1} a_k b_k.$$

The work is approximately

$$2K - 1$$

floating point operations, consisting of K multiplications and $K - 1$ additions.

Matrix Multiplication

For matrices

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

matrix multiplication is

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

The total number of floating point operations is

$$2MNK.$$

For square matrices where $M = N = K = n$, this becomes

$$2n^3.$$

Naive Matrix Multiplication Arithmetic Intensity

A naive one-output-per-invocation matrix multiplication kernel performs $2K$ FLOPs per output element. It loads approximately K values from A , K values from B , and writes one value to C .

For FP32 values, the approximate bytes moved per output are

$$4K + 4K + 4 = 8K + 4.$$

Thus, the arithmetic intensity is

$$\text{AI}_{\text{naive}} = \frac{2K}{8K + 4}.$$

For large K ,

$$\text{AI}_{\text{naive}} \approx \frac{1}{4}.$$

This is low, so the naive kernel is likely to be memory-bandwidth limited.

Tiled Matrix Multiplication

Tiling improves reuse by loading blocks of A and B into workgroup memory. If the tile size is T , a tile multiplication performs approximately

$$2T^3$$

FLOPs and loads approximately

$$2T^2$$

elements. For FP32 data, the arithmetic intensity is approximately

$$\text{AI}_{\text{tile}} = \frac{2T^3}{2T^2 \cdot 4} = \frac{T}{4}.$$

This shows why tiling matters. Increasing T increases reuse and arithmetic intensity. However, larger tiles consume more workgroup memory and can reduce occupancy.

Roofline Model

The roofline model bounds achieved performance by compute throughput and memory bandwidth:

$$P \leq \min(P_{\text{peak}}, B_{\text{mem}} \cdot \text{AI}).$$

Here, P is achieved performance, P_{peak} is peak compute throughput, B_{mem} is memory bandwidth, and AI is arithmetic intensity.

If

$$B_{\text{mem}} \cdot \text{AI} < P_{\text{peak}},$$

the kernel is bandwidth-bound. If

$$B_{\text{mem}} \cdot \text{AI} \geq P_{\text{peak}},$$

the kernel may become compute-bound.

Code Walkthrough

Representative Host Program

The following code is a representative version of the `gpu.cpp` programming style. It is intentionally minimal and focuses on the conceptual structure.

```
#include "gpu.h"
#include <vector>
#include <iostream>

int main() {
    auto ctx = gpu::create_context();

    std::vector<float> x = {1.0f, 2.0f, 3.0f, 4.0f};
    std::vector<float> y(x.size(), 0.0f);

    auto x_gpu = gpu::create_tensor(ctx, x);
    auto y_gpu = gpu::create_tensor<float>(ctx, x.size());

    auto kernel = gpu::create_kernel(ctx, R"(
```

```

    @group(0) @binding(0)
    var<storage, read> x : array<f32>;

    @group(0) @binding(1)
    var<storage, read_write> y : array<f32>;

    @compute @workgroup_size(256)
    fn main(@builtin(global_invocation_id) gid : vec3<u32>) {
        let i = gid.x;
        y[i] = x[i] * 2.0;
    }
)", {x_gpu, y_gpu});

gpu::dispatch(kernel, gpu::Workgroups{1, 1, 1});
gpu::wait(ctx);

gpu::copy_to_cpu(ctx, y_gpu, y);

for (float v : y) {
    std::cout << v << std::endl;
}
}

```

The important steps are:

- create the GPU context;
- allocate input and output buffers;
- create a WGSL kernel;
- bind buffers to the kernel;
- dispatch GPU work;
- wait only when the result is needed;
- copy the output back to CPU memory.

Bounds-Safe WGSL Map Kernel

```

struct Params {
    n : u32,
};

@group(0) @binding(0)
var<storage, read> x : array<f32>;

@group(0) @binding(1)
var<storage, read_write> y : array<f32>;

```

```

@group(0) @binding(2)
var<uniform> params : Params;

@compute @workgroup_size(256)
fn main(@builtin(global_invocation_id) gid : vec3<u32>) {
    let i = gid.x;

    if (i >= params.n) {
        return;
    }

    y[i] = 2.0 * x[i];
}

```

The bounds check prevents out-of-range memory accesses when the dispatch launches more invocations than logical elements.

CUDA Equivalent

```

__global__ void scale_kernel(const float* x, float* y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i >= n) {
        return;
    }

    y[i] = 2.0f * x[i];
}

```

The CUDA index calculation and the WebGPU global invocation ID serve the same role: they identify the logical element handled by the current execution lane.

Naive Matrix Multiplication Kernel

```

@group(0) @binding(0)
var<storage, read> A : array<f32>;

@group(0) @binding(1)
var<storage, read> B : array<f32>;

@group(0) @binding(2)
var<storage, read_write> C : array<f32>;

struct Params {
    M : u32,

```

```

    N : u32,
    K : u32,
};

@group(0) @binding(3)
var<uniform> params : Params;

@compute @workgroup_size(16, 16, 1)
fn main(@builtin(global_invocation_id) gid : vec3<u32>) {
    let col = gid.x;
    let row = gid.y;

    if (row >= params.M || col >= params.N) {
        return;
    }

    var acc : f32 = 0.0;

    for (var k : u32 = 0u; k < params.K; k = k + 1u) {
        let a = A[row * params.K + k];
        let b = B[k * params.N + col];
        acc = acc + a * b;
    }

    C[row * params.N + col] = acc;
}

```

This kernel assigns one output element to each invocation. It is simple but inefficient because it repeatedly loads values from global memory without explicit tile reuse.

Tiled Matrix Multiplication Sketch

```

const TILE : u32 = 16u;

var<workgroup> As : array<array<f32, 16>, 16>;
var<workgroup> Bs : array<array<f32, 16>, 16>;

@compute @workgroup_size(16, 16, 1)
fn main(
    @builtin(global_invocation_id) gid : vec3<u32>,
    @builtin(local_invocation_id) lid : vec3<u32>
) {
    let local_col = lid.x;
    let local_row = lid.y;

```

```

let global_col = gid.x;
let global_row = gid.y;

var acc : f32 = 0.0;

for (var tile : u32 = 0u; tile < params.K; tile = tile + TILE) {
    As[local_row][local_col] =
        A[global_row * params.K + tile + local_col];

    Bs[local_row][local_col] =
        B[(tile + local_row) * params.N + global_col];

    workgroupBarrier();

    for (var k : u32 = 0u; k < TILE; k = k + 1u) {
        acc = acc + As[local_row][k] * Bs[k][local_col];
    }

    workgroupBarrier();
}

C[global_row * params.N + global_col] = acc;
}

```

The first barrier ensures all tile loads finish before computation begins. The second barrier ensures the tile is not overwritten before every invocation finishes using it.

Performance Intuition

Why Tiling Improves Matrix Multiplication

Naive matrix multiplication has poor reuse. Many invocations repeatedly load the same A and B elements from global memory. Tiling loads blocks into workgroup memory once and reuses them across many multiply-add operations.

The lecture describes a performance ladder similar to classic CUDA optimization work:

1. naive one-output-per-invocation matrix multiplication;
2. shared-memory or workgroup-memory tiling;
3. more computation per invocation;
4. one-dimensional block tiling;
5. two-dimensional block tiling;

6. FP16 variants;
7. loop unrolling.

The major lesson is that many CUDA performance ideas transfer to WebGPU and WGSL because the underlying hardware constraints remain similar.

Why Fast Iteration Matters

The speaker emphasizes fast compile and edit cycles. Building a large native WebGPU implementation from source repeatedly would make experimentation slow. `gpu.cpp` avoids this by linking against shared libraries so that the user mostly recompiles their own small C++ code.

Fast iteration matters because GPU debugging is often experimental:

- modify the kernel;
- write diagnostic values to an output buffer;
- dispatch the kernel;
- inspect the output;
- repeat.

Debugging Strategy

GPU kernels are harder to debug than CPU code because ordinary step-through debugging is often unavailable or inconvenient. A practical debugging strategy is to write intermediate values into output buffers.

For example, to inspect global invocation IDs:

```
output[i] = f32(gid.x);
```

The program can then copy the buffer back or visualize it. Hot reloading makes this workflow more interactive.

Common Misconceptions

Misconception: WebGPU Is Only for Browsers

WebGPU can run inside browsers, but native implementations such as Dawn allow C++ programs to use WebGPU outside the browser.

Misconception: `gpu.cpp` Is a Full ML Framework

`gpu.cpp` is not a replacement for PyTorch, JAX, or TensorFlow. It is a lightweight direct-compute layer that can be used for ML kernels, simulations, educational tools, and bespoke local runtimes.

Misconception: Portability Guarantees Peak Performance

Portability does not guarantee peak vendor-specific performance. CUDA may expose NVIDIA-specific features earlier than WebGPU. WebGPU prioritizes broad support across devices and platforms.

Misconception: Workgroup Size Is Only Syntax

Workgroup size affects occupancy, memory behavior, scheduling, synchronization cost, and workgroup memory usage. It is a real performance parameter.

Practical Takeaways

- `gpu.cpp` wraps native WebGPU to make small C++ GPU programs easier to write.
- WebGPU can target native backends such as Metal, DirectX, and Vulkan.
- WGSL is the kernel language used for WebGPU compute.
- WebGPU workgroups are conceptually similar to CUDA thread blocks.
- WebGPU invocations are conceptually similar to CUDA threads.
- WebGPU workgroup memory is conceptually similar to CUDA shared memory.
- Dispatch is asynchronous, so synchronization should be explicit and minimized.
- Kernel creation should stay outside the hot path.
- Matrix multiplication is a useful test of whether a system can express serious GPU optimizations.
- Portable APIs may lag behind vendor-specific APIs for specialized features such as tensor cores.
- Browser deployment creates a path for interactive GPU education and local compute applications.

Project or Experiment Ideas

Vector Map Benchmark

Implement a vector map operation on CPU and with `gpu.cpp`. Sweep

$$N \in \{10^3, 10^4, 10^5, 10^6, 10^7\}.$$

Measure when GPU dispatch overhead is outweighed by parallel throughput.

Naive Versus Tiled Matrix Multiplication

Implement naive and tiled matrix multiplication. Measure achieved performance as

$$P = \frac{2MNK}{T_{\text{seconds}}}.$$

Compare tile sizes, matrix sizes, and data types.

Workgroup Size Sweep

Sweep workgroup sizes such as

$$B \in \{32, 64, 128, 256, 512\}.$$

Measure effective bandwidth:

$$B_{\text{effective}} = \frac{\text{bytes read} + \text{bytes written}}{T_{\text{seconds}}}.$$

Browser GPU Puzzle

Build a browser-based GPU puzzle where the user edits WGSL code and the browser runs the kernel locally. This demonstrates the educational value of WebGPU.

Hot-Reloaded Simulation

Build a particle simulation or double-pendulum simulation where GPU state persists while the kernel changes. This demonstrates runtime kernel replacement and persistent GPU buffers.

Final Mental Model

`gpu.cpp` is a lightweight bridge between ordinary C++ and portable GPU compute. It does not hide the GPU execution model. Instead, it removes enough boilerplate that the programmer can focus on buffers, kernels, workgroups, dispatch, memory reuse, and synchronization.

The overall stack is

$$\text{C++ application} \rightarrow \text{gpu.cpp} \rightarrow \text{WebGPU} \rightarrow \begin{cases} \text{Metal,} \\ \text{DirectX,} \\ \text{Vulkan.} \end{cases}$$

The device-side model is

WGSL kernel+workgroups+invocations+storage buffers+workgroup memory.

The final mental model is that `gpu.cpp` tries to make local GPU compute programmable, portable, and interactive without becoming a large framework. It gives researchers, educators, and systems programmers a small direct path to heterogeneous local GPUs.

Visual Concept

Draw a layered stack. At the top, place a C++ application using `gpu.cpp`. Below it, draw WebGPU. Below WebGPU, branch into Metal, DirectX, and Vulkan. At the bottom, draw Apple GPUs, Windows GPUs, Linux GPUs, and browser-accessible local GPUs. On the side, draw CUDA concepts mapped to WebGPU concepts: thread block to workgroup, thread to invocation, global memory buffer to storage buffer, and shared memory to workgroup memory.